1 Introduction to Software Engineering Management

Week 1 · Worksheet

Name _____ Date _____

Exercises

Show your work. Each exercise has a hint at the end: try without it first.

1. Spot the gap Warm-up · Lesson 1

A client says "make the report faster". List two questions you would ask before writing any code.

Hint: Ask what "faster" means for them: seconds, minutes, or a different data set?

2. Write a testable requirement Practice · Lesson 1

Rewrite "the login should be secure" as a requirement with a measurable condition.

Hint: Name the condition, the actor, and the expected result.

3. Pick a model Warm-up · Lesson 2

A government form has fixed legal rules that will not change for two years. Which process model fits best, and why?

Hint: Think about how much change you expect after the requirements are signed.

4. Draw your own baseline history Practice · Lesson 2

A team builds a chat app in three iterations: login, messages, and notifications. Name the baseline and increment after iteration 2.

Hint: A baseline is a working version; the increment is the difference from the previous one.

5. Find the boundary Warm-up · Lesson 3

A library asks for "a system for books". Write two assumptions or constraints that set the scope.

Hint: Say what is included and what is explicitly out.

6. Split the layers Practice · Lesson 3

For an online pharmacy, write one business, one functional, one non-functional, and one transition requirement.

Hint: Business = goal; functional = action; non-functional = quality with a number; transition = temporary.

7. Draw a context diagram in words Warm-up · Lesson 4

For a food delivery app, list the external actors and one data flow in and out for each.

Hint: Think of customers, restaurants, couriers, and the payment provider.

8. Design a state machine Challenge · Lesson 4

List the states of an online order and the event for each transition.

Hint: Start at Created; end at Delivered or Cancelled.

9. Inventory the configuration items Warm-up · Lesson 5

List five configuration items for a mobile app that are not source code.

Hint: Think about documents, tests, build settings, and releases.

10. Trace a bug to a baseline Challenge · Lesson 5

Production shows a bug that was not in the test build. Describe the four SCM questions you would answer to find the cause.

Hint: Identification, version control, change control, status accounting.

11. Fix the commit message Warm-up · Lesson 6

Rewrite these messages in the imperative style: "stuff", "Fixed the bug.", "Added a new function for sums".

Hint: Start with a verb, lower-case, no final full stop, about 50 characters.

12. Split a commit Practice · Lesson 6

One commit mixes a bug fix, a rename, and a new test. Describe how you would split it into three logical commits.

Hint: Stage related files together and commit each group in turn.

13. Predict the status Warm-up · Lesson 7

You edit a.txt, then run git add a.txt, then edit a.txt again. Is the second edit staged? What does git status show?

Hint: Staging captures the file at the moment you run git add.

14. Read the graph Practice · Lesson 7

Run git log --oneline --graph --all in a sandbox after Lab 2. Explain in one sentence why main and feature-sum point to the same commit.

Hint: What happens when the branch you merge into has no new commits?

15. Resolve by intent Challenge · Lesson 7

Two branches changed the same function: one renamed a parameter, the other added validation. Describe the resolution you want and the commands.

Hint: Keep both intents, then stage and commit.

16. Order the workflow Warm-up · Lesson 8

Put these in order: merge, open pull request, push feature branch, review and approve, create branch.

Hint: You cannot open a request before the branch exists on the remote.

17. Recover from a rejected push Challenge · Lesson 8

A teammate pushed to main while you were working. Your git push is rejected. Write the safe sequence of commands.

Hint: Fetch first, integrate, then push. Never force.

Quick check

Circle one answer.

18. Which item is a configuration item in software configuration management?

A) A requirements document B) The office coffee machine C) A personal calendar

19. Which statement about Git is correct?

A) Git is a distributed version control system; every clone holds the full history.

B) Git cannot record a commit without an internet connection to GitHub.

C) Git and GitHub are two names for the same tool.

20. In an iterative and incremental process, what does the difference between two successive baselines represent?

A) An increment B) A risk register C) The final delivery only

21. A teammate says the last release behaves incorrectly. Which question should you ask first?

A) Which configuration items and versions are in that baseline? B) Who has the fastest laptop?

C) Can we delete the repository and start again?

22. You want a teammate to check your login change before it reaches main. What do you do?

A) Push a feature branch and open a pull request for review

B) Push directly to main and ask for feedback afterwards C) Email a zip file of the project

23. Two branches changed the same line in app.js. The merge stops. What happened?

A) A merge conflict; a person must choose the final content B) Git deleted the repository

C) The remote is offline

24. git push is rejected because the remote has commits you do not have locally. What is the safest fix?

A) Run git fetch, integrate the remote changes (merge or rebase), then push again

B) Run git push --force to overwrite the remote C) Delete your local repository and clone again

25. git commit fails with "Author identity unknown". What should you do?

A) Set your name and email with git config --global user.name and user.email B) Delete the .git folder

C) Commit with git add -f

Answer key

1. Example: "Which report, and how slow is it now versus the target?" and "Who uses it, and how often?" Each answer turns a vague wish into a requirement you can test.
2. Example: "After 5 failed logins within 10 minutes, the account is locked for 15 minutes and the user receives an email." A tester can now check it.
3. A sequential (waterfall-style) process can fit, because requirements are stable and early mistakes would be expensive. An iterative process would also work if the team wants early feedback on the interface.
4. Baseline 2 is login plus messages, working and documented. The increment is the messaging feature compared with baseline 1.
5. Example: "Includes loans and returns for members." "Excludes buying new books and paying staff salaries." Clear limits prevent scope problems.
6. Business: "increase online sales by 20% this year." Functional: "customers upload a prescription photo." Non-functional: "orders are confirmed within 5 seconds." Transition: "import 10 years of customer records from the old system."
7. Customer: order in, status out. Restaurant: menu in, order out. Courier: location in, pickup task out. Payment provider: confirmation in, charge request out.
8. Created --pay--> Paid --pack--> Packed --ship--> Shipped --deliver--> Delivered. Created or Paid --cancel--> Cancelled. Each arrow is an event; each box is a state.
9. Examples: requirements specification, UI design files, test plan and test scripts, build configuration files, release notes, user manual.
10. Which exact versions are in production (identification)? What changed between the test build and production (version control)? Which changes were approved and by whom (change control)? Is the production build the one recorded in the baseline (status accounting)?
11. fix login redirect loop | fix bug where … (name the bug) | add sum function. Specific verbs help reviewers scan history quickly.
12. Stage the bug fix files, commit; stage the renamed files, commit; stage the test, commit. Each commit can now be reviewed or reverted alone.
13. No: the second edit is not staged. git status lists a.txt under both "Changes to be committed" (the first version) and "Changes not staged for commit" (the newer edit).
14. main had no new commits since feature-sum split, so Git moved the pointer forward (fast-forward) instead of creating a merge commit.
15. Edit the function so it uses the new parameter name and includes the validation. Remove all markers, run git add on the file, then git commit to finish the merge.
16. Create branch → push feature branch → open pull request → review and approve → merge.
17. git fetch origin, then git pull origin main (or git rebase origin/main), resolve any conflicts, run tests, then git push origin main.
18. **A** — Any artefact that is part of the product or its documentation can be a configuration item. Requirements are a core example.
19. **A** — Each clone has the full repository and history, so you can commit offline.
20. **A** — By definition, the difference between two baselines is an increment.
21. **A** — Before debugging, identify exactly which versions were released. SCM makes this traceable.
22. **A** — A feature branch with a pull request lets reviewers see the diff and approve before merging.
23. **A** — Git cannot choose between two different edits to the same lines, so it marks the conflict for you to resolve.

24. **A** — Integrate the remote work first. A forced push can erase a teammate's commits and should be avoided on shared branches.
25. **A** — Git stores the author name and email in every commit, so both must be configured first.